PROOF-CARRYING DEBUGGER

Buggy

Usage & Sandbox Guide — how to run it on any project,
a real RAG-chatbot walkthrough, and how the execution sandbox works

Version 0.1.0 · MIT License · Node ≥ 18
Applies to TypeScript / JavaScript projects

1. What Buggy is

Buggy is a multi-agent **proof-carrying debugger**. Instead of flagging code that "looks risky," it runs a four-stage pipeline that finds a concrete input, proves the bug reproduces, proposes candidate fixes, and rejects fixes that only paper over the failing test.

Parse → **Prove** → **Repair** → **Classify**, coordinated by an orchestrator, and exposed three ways:

- **CLI** — `buggy` for local/CI use.
- **MCP server** — `buggy-mcp`, so AI IDEs (Kiro, Cursor, Windsurf) call it as tools.
- **Programmatic API** — the `ProofDebugger` class for scripts and integrations.

It also ships a Kiro integration (hooks + steering) and an **experience memory ("Watchlist")** that records what worked and what failed on every investigation, so results improve with each iteration.

> **What makes it different:** a *proof-of-failure certificate* (a real triggering input, the faulty output, and the violated postcondition, verified for admissibility, soundness, and reproducibility) plus an *overfitting classifier* that throws out patches which only match the test.

2. Language support (read this first)

> **Today, Buggy runs on TypeScript and JavaScript only.** The parser and the execution engine are wired for the TS/JS grammar and a Node runtime. Python and other languages are **not yet supported** by the running pipeline, even though the config file lists them.

Why, precisely:

Parser	The parser loads the Tree-sitter **TypeScript grammar unconditionally**. It does not switch grammars based on `language` in your config, so a `.py` file would be parsed with TS rules and produce mostly error nodes.
Execution / proving	The engine proves bugs by **running the function in a Node child process** (it strips TypeScript to JavaScript and executes it). There is no Python interpreter path.
Config fields	`language`, `parser.command`, and `sandbox.runtime` are accepted and validated, but the parser and executor do not act on them yet.

> **For a RAG chatbot:** if your codebase is **TypeScript/JavaScript** (LangChain.js, LlamaIndex.TS, Vercel AI SDK), Buggy works today — this guide's walkthrough is real and runnable. If your RAG chatbot is **Python** (LangChain, LlamaIndex, FastAPI), Buggy cannot analyze it yet. Do not advertise Python support.

What adding Python would take

1. Add the `tree-sitter-python` grammar and make the parser select the grammar from `config.language`.
2. Add a Python execution runner in the sandbox/executor (spawn `python`, marshal inputs, capture return/exception), parallel to the current Node runner.
3. Extend the TypeScript-stripping step (which only applies to JS/TS) so it is skipped or replaced for Python.

This is genuine engineering work, not a config switch.

3. Install & set up on any (TS/JS) project

```
# 1. Install
npm install buggy          # or: npm install -g buggy for the CLI

# 2. Initialize in your project root
cd /path/to/your/project
npx buggy init             # creates .debugger.yaml and .debugger/

# 3. Ignore the working directory
echo ".debugger/" >> .gitignore
```

Configure `.debugger.yaml`

```
language: typescript
parser:   { command: tree-sitter-typescript }
lsp:      { command: typescript-language-server }   # optional
sandbox:  { runtime: node, memory_limit_mb: 512, timeout_seconds: 60 }
oracles:  { timeout_threshold_seconds: 10, crash_detection: true,
            overflow_detection: true, determinism_check_count: 5 }
probe:    { search_budget: 100, max_refinement_iterations: 10 }
watchlist:{ enabled: true, scope: layered }   # experience memory (see §8)
```

> The `watchlist.scope` defaults to `layered`: raw episodes stay local, verified lessons are shared with your team via committed steering, and generalized lessons corroborated across ≥2 projects promote to a personal global store. Set `scope: local` for sensitive repos.

4. The pipeline, stage by stage

Stage	What happens	Output
Parse	Fault-tolerant Tree-sitter CST, optional LSP symbol resolution, call graph.	CST + errors, graph in SQLite
Prove	Backward slice, spec refinement (PROBE loop), specification-aware fuzzing that runs the function on generated inputs looking for a postcondition violation.	Proof-of-failure certificate, or `unconfirmed`
Repair	AST-aware candidate patches from multiple strategies (guards, expression/return correction, etc.).	≥3 candidate patches
Classify	66-dimension AST-difference vector scored for overfitting; approve below threshold, reject above.	Approved / rejected patches

Terminal statuses: `confirmed_and_repaired` (bug proven + at least one non-overfit fix), `confirmed_no_repair` (bug proven, all fixes rejected), `unconfirmed` (no bug proven), `halted`.

5. Walkthrough: a RAG chatbot (TypeScript)

A retrieval-augmented chatbot lives or dies on its retrieval math. The classic production incidents are silent: a NaN similarity from an empty embedding, a crash on an empty document set, an infinite loop in a chunker. Here is Buggy run against a real TS RAG pipeline (embeddings, chunking, retrieval). **Every result below is actual output.**

The code under test

```
// embeddings.ts
export function cosineSimilarity(a: number[], b: number[]): number {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) { dot += a[i]*b[i]; normA += a[i]**2; normB += b[i]**2; }
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));   // zero vector -> 0/0 = NaN
}
export function averageEmbeddings(embeddings: number[][]): number[] {
  const dim = embeddings[0].length;                     // [] -> undefined.length -> throws
  /* ... */
}
// retriever.ts
export function bm25Score(query: string[], document: string[], avgDocLength: number, ...): number {
  /* ... document.length / avgDocLength ... */          // avgDocLength 0/null -> NaN
}
```

Run it

```
buggy investigate cosineSimilarity --file src/embeddings.ts
buggy investigate averageEmbeddings --file src/embeddings.ts
buggy investigate bm25Score        --file src/retriever.ts
```

Real results

```
cosineSimilarity   status=confirmed_no_repair
  proof.test_input  = [[], []]          # two empty embedding vectors
  observed_output   = NaN
  violated          = !isNaN(result)
  patches           = 0 approved, 3 rejected (all flagged overfit)

bm25Score          status=confirmed_no_repair
  proof.test_input  = [[""], [""], null]  # avgDocLength = null
  observed_output   = NaN
  violated          = !isNaN(result)
  patches           = 0 approved, 3 rejected

averageEmbeddings  status=confirmed_no_repair
  proof.test_input  = [[]]              # empty embedding set
  observed_output   = (threw)
  violated          = function must not throw:
                      TypeError: Cannot read properties of undefined (reading 'length')
  patches           = 0 approved, 3 rejected
```

What this proves in practice: before your chatbot ships, Buggy hands you the exact inputs that break retrieval — an empty embedding makes similarity `NaN` (every document ties, ranking is garbage), and an empty result set crashes the aggregator. These are certified with a reproducing input, not guessed.

Honest note on the fixes: in all three cases every candidate patch was classified as *overfit* and rejected, so the status is `confirmed_no_repair`. Buggy is deliberately conservative: it would rather hand you a proven bug than auto-apply a fix that only satisfies the one failing input. You then write the guard (e.g. return 0 for a zero-norm vector) with the exact trigger in hand.

Same thing over MCP (what Kiro/Cursor call)

```
tool: buggy_investigate
args: { "function_id": "cosineSimilarity", "file_path": "src/embeddings.ts",
        "postconditions": ["!isNaN(result)", "isFinite(result)"] }

-> { "status": "confirmed_no_repair",
     "proof": { "test_input": [[],[]], "observed_output": null,
                "violated_postcondition": "!isNaN(result)" },
     "approved_patches": 0, "rejected_patches": 3 }
```

Programmatic API

```
import { ProofDebugger } from 'buggy';
const dbg = new ProofDebugger({ projectRoot: process.cwd() });
await dbg.initialize();
const report = await dbg.investigate({
  functionId: 'cosineSimilarity', filePath: 'src/embeddings.ts',
  specification: { postconditions: ['!isNaN(result)', 'isFinite(result)'],
                   parameters: [{name:'a',type:'number[]'},{name:'b',type:'number[]'}],
                   return_type: 'number' },
});
console.log(report.status, report.proof?.test_input);   // confirmed_no_repair [ [], [] ]
await dbg.shutdown();
```

The memory makes the next run smarter

```
tool: buggy_recall  args: { "function_id": "cosineSimilarity" }
-> { "seen_before": true,
     "lessons": [ { "tier": "project", "title": "nan_result in cosineSimilarity",
                    "what_failed": ["Overfitting risk: ..."], "trigger_shape": "[[],[]]" } ] }
```

With the *recall-first* hook installed, the next time anyone asks Kiro to touch `cosineSimilarity`, Kiro is reminded of the proven `NaN` at `[[],[]]` and which fix shapes were rejected — so it writes the zero-vector guard directly.

6. The execution sandbox — how it actually works

To prove a bug, Buggy must *run* your function on generated inputs. Here is exactly how that execution is contained today, stated plainly so you can represent it accurately.

What runs your code

Mechanism	A **forked Node.js child process**. Buggy writes a small generated runner script to a temp directory, forks it, calls your function, and returns the result to the parent.
Result channel	Results come back over a dedicated **IPC channel**, so anything the function prints to stdout cannot corrupt the result payload.
Timeout	A parent-managed **wall-clock timeout** (default 5s; influenced by `oracles.timeout_threshold_seconds`). On timeout the child is `SIGKILL`ed — this is how infinite-loop bugs (e.g. a chunker with `overlap ≥ chunkSize`) are caught.
Scratch space	Runner scripts are written under the OS temp dir (`<tmp>/buggy-exec`) and deleted after each run.
Edge-value fidelity	`NaN`, `Infinity`, `-Infinity`, and `undefined` are preserved through sentinels, so the very edge cases that cause bugs are tested faithfully (plain JSON would lose them).
TS handling	TypeScript is stripped to plain JavaScript before execution.
Determinism check	The function can be run N times on the same input to distinguish real bugs from flakes.

Oracles (what counts as a failure)

- **Crash** — an uncaught exception (e.g. the `averageEmbeddings([])` `TypeError` above).
- **Timeout** — exceeds the wall-clock budget (infinite loop / pathological input).
- **Postcondition violation** — output breaks a declared spec (e.g. `!isNaN(result)`).
- **Non-determinism** — differing outputs across identical runs.
- **Overflow** — integer/numeric overflow detection.

> **Isolation reality — important for security claims.** The active sandbox is **process-level, not VM-level**. The forked child shares the host filesystem, network, and user privileges of the parent Node process. Memory limits are *reported, not OS-enforced*. This is fine for analyzing *your own* code. Do **not** point it at untrusted third-party code expecting containment, and do not market it as "hardware isolation."

> **Firecracker microVM — designed, not yet active.** The codebase includes a microVM isolation layer (per-VM memory caps, network egress policy, syscall allow-list, OAP execution passports, a circuit breaker, and a snapshot pool for fast boot). It is **not wired into the default pipeline** — the sandbox agent reports itself unavailable and the subprocess executor is used instead. Treat microVM isolation as roadmap, and enable/validate it before making isolation guarantees.

7. Interface reference

CLI

Command	Purpose
`buggy init`	Create `.debugger.yaml` + `.debugger/`.
`buggy analyze <file>`	Parse a file; list functions and syntax errors.
`buggy investigate <fn> --file <path>`	Run the full pipeline on a function.
`buggy status <id>`	Progress of a running/finished investigation.
`buggy halt <id>`	Stop an investigation, keep partial results.

Global flags: `--json` (machine-readable), `--verbose`.

MCP tools (7)

Tool	Purpose
`buggy_init`	Initialize the debugger for a project.
`buggy_analyze`	Parse a file, return CST analysis + function list.
`buggy_investigate`	Run the full Parse→Prove→Repair→Classify pipeline.
`buggy_status`	Investigation status.
`buggy_query_graph`	Query the semantic graph (callees, node, file graph).
`buggy_list_functions`	List function declarations in a file.
`buggy_recall`	Recall prior Watchlist lessons before editing/fixing code.

API (`ProofDebugger`)

`initialize`, `parse`, `investigate`, `getStatus`, `halt`, `queryCallees`, `queryNode`, `queryFileGraph`, `getConfig`, `recall`, `watchlistStats`, `shutdown`.

8. Watchlist — the experience memory

Every investigation writes one **episode** (what was attempted, what worked, what failed and how). Verified, proof-backed episodes distill into **lessons** that are fed back before the next change.

Scope	Lives in	Shared with
local	`.debugger/` (git-ignored)	nobody — raw episodes
team	`.kiro/steering/buggy-watchlist.md` (committed)	teammates on the repo
global	`~/.buggy/` + user-level steering	all your projects (sanitized, ≥2-project corroboration)

- **Recall**: `buggy_recall` and `ProofDebugger.recall()` merge project + global lessons, project winning on conflict.
- **Safety**: only proof-backed lessons promote; the global tier is sanitized of code, paths, and literal values; investigations never modify your source files.
- **Kiro**: the *recall-first* hook consults the memory before edits; committed team steering means teammates inherit every lesson.

9. Limitations to represent honestly

Area	Reality today
Languages	TypeScript/JavaScript only. Python and others are not analyzed yet (§2).
Isolation	Process-level (forked Node subprocess), not VM-level. Firecracker microVM is present but inactive (§6).
Auto-fix	Proposes and screens fixes; frequently rejects all candidates as overfit, yielding `confirmed_no_repair`. Best framed as "proves bugs and screens fixes," not "auto-fixes."
Patch validation	The compile/emulate/test filtering pipeline exists but is not on the default investigate path; candidates are screened for overfitting, not compile/test-verified.
Best-fit inputs	Scalar/numeric functions prove most readily; array/object inputs may need tighter preconditions or return `unconfirmed`.

> **Rock-solid today:** TS/JS parsing, bug proving with reproducing certificates, overfitting classification, the CLI / MCP / API surfaces, the Kiro hooks & steering, and the full Watchlist memory — all test-backed (900+ passing tests).
